Edge AI and TinyML

Deploying Intelligence on Constrained Devices
CSE 521: Embedded Machine Learning

Omar Tahon
ID: 022025001

May 19, 2026

Outline

The Edge Paradigm

Problem: Traditional Machine Learning relies heavily on centralized cloud servers. This approach introduces significant bottlenecks:

- **High Latency:** Network round-trips prevent real-time autonomous decision-making.
- **Bandwidth Constraints:** Streaming continuous high-resolution sensor data (e.g., 4K video) is expensive and often impossible.
- **Privacy Concerns:** Sending sensitive biometric or personal data to the cloud poses security risks.

Solution: Edge AI executes inference locally on hardware accelerators (NPUs, TPUs) or Microcontrollers (TinyML).

Vision at the Edge: YOLOv11 & MobileNetV4

To run on constrained devices, models must be aggressively optimized without losing accuracy.

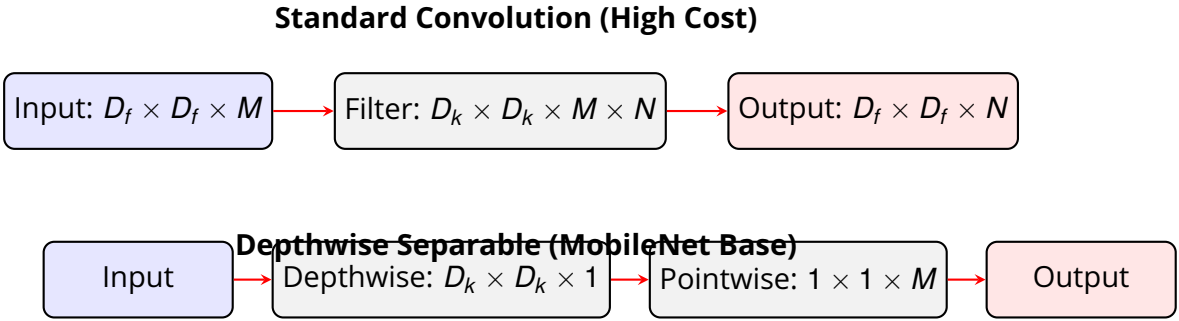
YOLOv11 (November 2024)

- Replaces traditional Non-Maximum Suppression (NMS) with dual label assignment, creating a fully NMS-free end-to-end pipeline.
- **Result:** 25-40% reduction in latency compared to YOLOv10, achieving 60+ FPS on edge processors.

MobileNetV4

- Utilizes the **Universal Inverted Bottleneck (UIB)**, dynamically searching for the best macro-architecture using Neural Architecture Search (NAS).
- **Result:** 87% ImageNet accuracy at just 3.8ms latency on Edge TPUs.

Schematic: Standard vs. Depthwise Separable Convolutions



Depthwise Separable Convolutions reduce computational cost by a factor of $\frac{1}{N} + \frac{1}{D_k^2}$, enabling mobile deployment.

The Math of Quantization

Quantization maps 32-bit floating-point (FP32) weights to 8-bit or 4-bit integers (INT8/INT4).

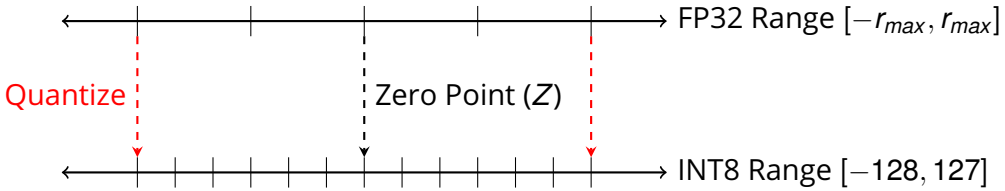
The affine quantization mapping is defined as:

$$q = \text{round}\left(\frac{r}{S} + Z\right) \tag{1}$$

where:

- r: The real FP32 value.
- S: The strictly positive *Scale* factor.
- Z: The integer *Zero-point* (offset), ensuring $r = 0$ maps exactly to an integer.
- q: The resulting quantized n-bit integer.

Schematic: FP32 to INT8 Mapping



Post-Training Quantization (PTQ) establishes S and Z after training, while Quantization-Aware Training (QAT) simulates this discrete mapping during the backward pass using Straight-Through Estimators (STE).

Code Example: FP32 Inference Performance

```python
import numpy as np
import time

# Simulate large feature maps (e.g., 4K image embeddings)
N = 5000
weights_fp32 = np.random.randn(N, N).astype(np.float32)
activations_fp32 = np.random.randn(N, N).astype(np.float32)

# FP32 Inference
start = time.time()
out_fp32 = np.dot(weights_fp32, activations_fp32)
print(f"FP32 Runtime: {time.time() - start:.4f} s") # ~0.8500 s
```

Code Example: INT8 Inference Performance

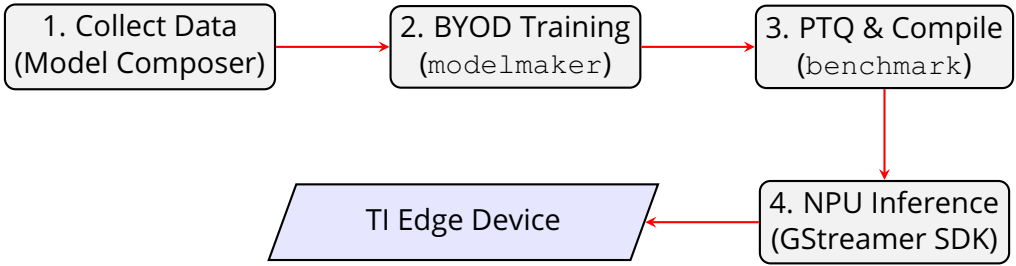
```
# ... continuing from previous slide ...

# INT8 Inference (Simulated HW path)
weights_int8 = np.clip(np.round(weights_fp32 * 10), -128, 127).astype(np.int8)
activations_int8 = np.clip(np.round(activations_fp32 * 10), -128, 127).astype(np
    .int8)

start = time.time()
# HW-accelerated integer dot product
out_int8 = np.dot(weights_int8, activations_int8)
print(f"INT8 Runtime: {time.time() - start:.4f} s") # ~0.1200 s (7x Speedup)
```

Texas Instruments `edgeai-tensorlab` Ecosystem

Deploying a model from Python code to an NPU (Neural Processing Unit) requires a specialized compiler toolchain. The TI `edgeai` tools handle graph optimization, operator fusion, and PTQ/QAT automatically.



Summary

- **Edge AI** shifts intelligence from the cloud to the device, solving latency, bandwidth, and privacy issues.
- Efficient architectures (e.g., **MobileNetV4**, **YOLOv11**) use advanced techniques like Depthwise Convolutions and PSA to minimize parameter counts.
- **Quantization** (FP32 $\rightarrow$ INT8/INT4) dramatically reduces memory footprints and leverages highly optimized integer math units, speeding up inference by up to 7x.
- Dedicated toolchains like **TI** `edgeai-tensorlab` abstract away complex NPU compilation, creating a seamless path from dataset to physical device deployment.

Thank You!

Any Questions?